Les tableaux à deux dimensions (Matrice):

Exercice1:

Ecrire un programme qui lit les dimensions L et C d'un tableau T à deux dimensions du type **int** (dimensions maximales: 50 lignes et 50 colonnes). Remplir le tableau par des valeurs entrées au clavier et afficher le tableau ainsi que la somme de chaque ligne et de chaque colonne en n'utilisant qu'une variable d'aide pour la somme.

Solution :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 int T[50][50]; /* tableau donné    */
 int L, C;      /* dimensions       */
 int I, J;      /* indices courants */
 long SOM;  /* somme des éléments - type long à cause   */
            /* de la grandeur prévisible des résultats. */

 /* Saisie des données */
 printf("Nombre de lignes   (max.50) : ");
 scanf("%d", &L );
 printf("Nombre de colonnes (max.50) : ");
 scanf("%d", &C );
 for (I=0; I<L; I++)
```

```
   for (J=0; J<C; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%d", &T[I][J]);
       }
/* Affichage du tableau */
printf("Tableau donné :\n");
for (I=0; I<L; I++)
   {
    for (J=0; J<C; J++)
         printf("%7d", T[I][J]);
    printf("\n");
   }
/* Calcul et affichage de la somme des lignes */
for (I=0; I<L; I++)
    {
     for (SOM=0, J=0; J<C; J++)
           SOM += T[I][J];
     printf("Somme - ligne %d : %ld\n",I,SOM);
    }
/* Calcul et affichage de la somme des colonnes */
for (J=0; J<C; J++)
    {
```

```
    for (SOM=0, I=0; I<L; I++)
        SOM += T[I][J];
    printf("Somme - colonne %d : %ld\n",J,SOM);
   }
 return 0;
}
```

Exercice2:

Ecrire un programme qui transfère un tableau M à deux dimensions L et C (dimensions maximales: 10 lignes et 10 colonnes) dans un tableau V à une dimension L*C.

Solution :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 int M[10][10]; /* tableau à 2 dimensions */
 int V[100];    /* tableau à 1 dimension  */
 int L, C;      /* dimensions       */
 int I, J;      /* indices courants */

 /* Saisie des données */
 printf("Nombre de lignes   (max.10) : ");
 scanf("%d", &L );
 printf("Nombre de colonnes (max.10) : ");
```

```
scanf("%d", &C );
for (I=0; I<L; I++)
   for (J=0; J<C; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%d", &M[I][J]);
       }
/* Affichage du tableau 2-dim */
printf("Tableau donné :\n");
for (I=0; I<L; I++)
   {
    for (J=0; J<C; J++)
         printf("%7d", M[I][J]);
    printf("\n");
   }
/* Transfer des éléments ligne par ligne */
for (I=0; I<L; I++)
     for (J=0; J<C; J++)
           V[I*C+J] = M[I][J];
 /* Affichage du tableau 1-dim */
printf("Tableau résultat : ");
for (I=0; I<L*C; I++)
    printf("%d ", V[I]);
```

```
 printf("\n");
 return 0;
}
```

Exercice 3:

Ecrire un programme C qui multiplie une matrice carré d'entiers M(N*N) par un réel X donné.

- Les éléments de la matrice A seront multipliés par X.
- Le résultat de la multiplication sera mémorisé dans une deuxième matrice A qui sera ensuite affichée.

Solution :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 float A[50][50]; /* matrice donnée   */
 float B[50][50]; /* matrice résultat */
 int N, M;        /* dimensions des matrices */
 int I, J;        /* indices courants */
 float X;         /* multiplicateur   */
 /* Saisie des données */
 printf("Nombre de lignes   (max.50) : ");
 scanf("%d", &N );
```

```
printf("Nombre de colonnes (max.50) : ");
scanf("%d", &M );
for (I=0; I<N; I++)
   for (J=0; J<M; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%f", &A[I][J]);
       }
printf("Multiplicateur X : ");
scanf("%f", &X );
/* Affichage de la matrice */
printf("Matrice donnée :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<M; J++)
         printf("%10.2f", A[I][J]);
    printf("\n");
   }
/* Affectation du résultat de la multiplication à B */
for (I=0; I<N; I++)
    for (J=0; J<M; J++)
         B[I][J] = X*A[I][J];
 /* Edition du résultat */
```

```
printf("Matrice résultat :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<M; J++)
        printf("%10.2f", B[I][J]);
    printf("\n");
   }
return 0;
}
```

Les chaines de caractères :

Exercice 1 :

```
#include <stdio.h>
Int main()
{
 /* Déclarations */
 char CH[100]; /* chaîne à convertir */
 int I;        /* indice courant     */

 /* Saisie de la chaîne */
 printf("Entrez la chaine à convertir : ");
 gets(CH);
 /* Conversion de la chaîne */
 for (I=0; CH[I]; I++)
```

```
    {
     if (CH[I]>='A' && CH[I]<='Z')
                CH[I] = CH[I]-'A'+'a';
     else if (CH[I]>='a' && CH[I]<='z')
                CH[I] = CH[I]-'a'+'A';
    }
  /* Affichage de la chaîne convertie */
 printf("Chaine convertie : %s\n", CH);
 return 0;
}
```

Exercice 2

```
#include <stdio.h>
#include <string.h>
Int main()
{ /* Déclarations */
char Mot[20]; /* chaine donnée */
int N, Pal=1; /* dimension */
int I, J; /* indices courants */
/* Saisie des données */
printf("Donner un mot(max.20) : ");
gets(Mot) ;
N = strlen(Mot) ;
```

```
I=0 ;
J=N-1;
while((I<N/2) && (Pal ==1))
{
if(Mot[I] != Mot[J])
Pal=0 ;
I++ ;
J-- ;
}
if(Pal==1)
    printf("\n Chaine palindrome");
else
 printf("\n chaine non palindrome");
return 0;
}
```

Exercice 3 :

```
#include <stdio.h>
#include <string.h>
int main()
{
 char M1[100];
 char M2[100];
 int n=0;
```

```
 printf("veuillez enter un mot: ");
 gets(M1);
 printf("veuillez enter un mot: ");
 gets(M2);
 if(strlen(M1)!=strlen(M2)){
    printf("les deux mots n'ont pas la meme longeur");
 }
 else{
    for(int i=0; i<strlen(M1); i++){
        if(M1[i]!=M2[i])
            n++;
    }
    printf("les distances de hamming entre les deux mots est:
%d",n);
 }
 return 0;
}
```